Introduction

Kafka Transactions guarantee that when a message is received, processed, and resulting message or messages are published, that these three stages together only happen once. This is termed ‘exactly-once messaging’. With the Kafka Streams API where messages are streamed into the application, Kafka Transactions can likewise be enabled to ensure each message is processed end to end exactly once, and that persisted state is durable and consistent.

This is one of a series of articles covering different aspects of Kafka Streams. Jump to the 'More On Kafka Streams...' section below to explore.

Kafka Transactions

Kafka Transactions are covered in detail in the article Kafka Transactions: Part 1 - Exactly-Once Messaging. It covers how they work with an application using the Kafka Consume & Produce APIs, the semantics around the term ‘exactly-once messaging’, and the application behaviour in failure scenarios when Transactions are enabled.

This article examines using Kafka Transactions with the Kafka Streams API. Kafka Streams is built on top of the Kafka Consume & Produce APIs, so with Kafka Transactions enabled it attains the same transactional behaviour and guarantees. However under the hood there is more going on with the Kafka Streams application. With stateful processing there is the additional concern of ensuring that application state is resilient to failure scenarios. Following a failure messages are guaranteed not to be lost. But messages must also be guaranteed to not result in duplicate writes to the state store.

Transactional Flow

The following diagram illustrates a Kafka Streams application consuming a batch of events, writing state to the state store, and producing outbound events. The flow is wrapped in a Kafka Transaction, so the application uses the Transaction Coordinator which lives on the broker to coordinate the transaction as it progresses.

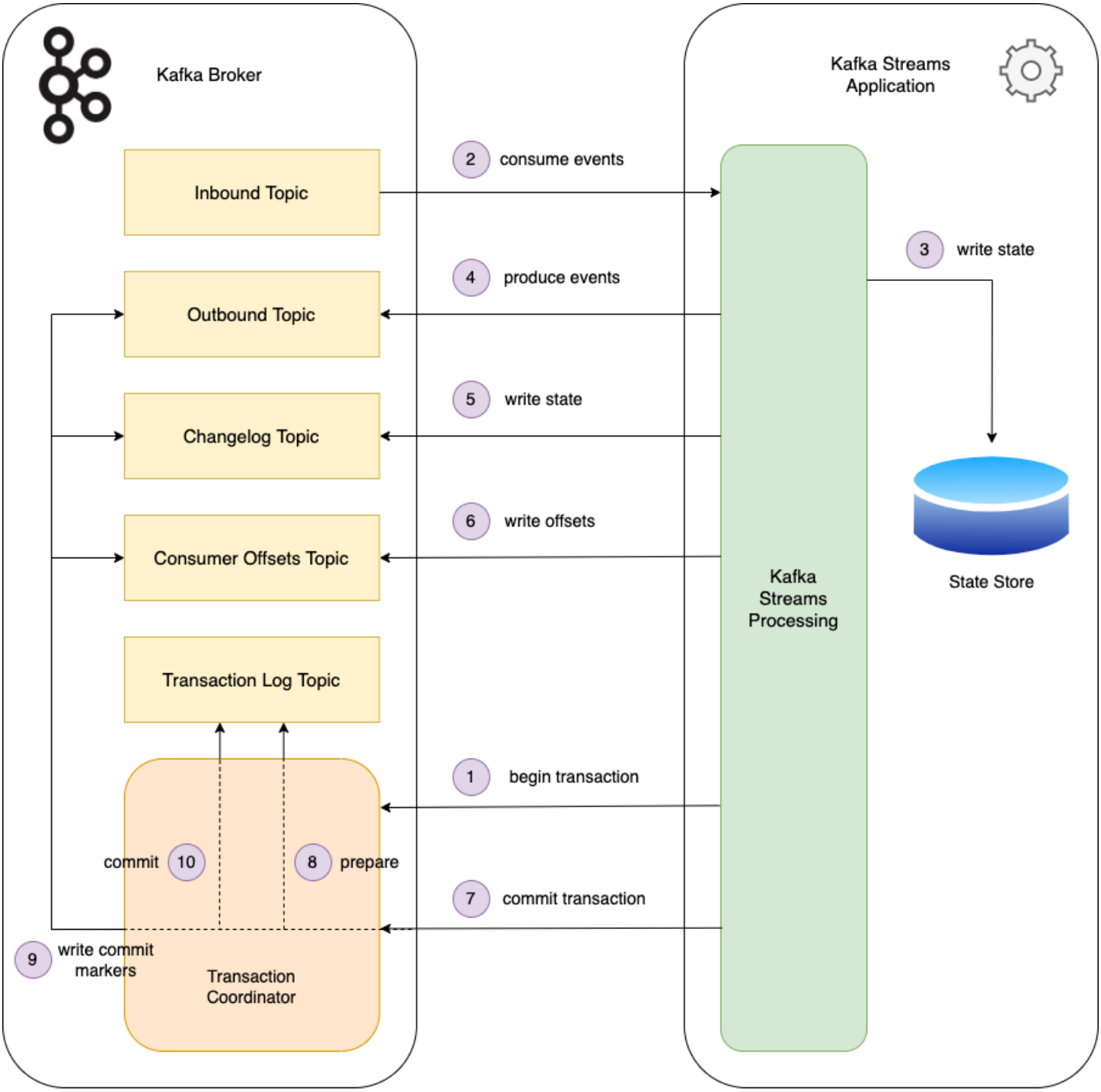

Figure 1: Transactional Kafka Streams flow - component diagram

1. The Kafka Streams library sends a *begin transaction* request to the Transaction Coordinator informing it that a transaction has begun.

2. A batch of events are consumed from the inbound topic and processed by the source processor to begin processing.

3. The state being captured is written to the local persistent state store (RocksDB by default).

4. Any sink processors in the processor topology write their resulting events to outbound topics.

5. The changelog topic, which backs the state store, is written with the state change.

6. The consumer offset topic is written with the offsets of the processed events, marking their processing as complete.

7. The Kafka Streams library sends a *commit transaction* request to the Transaction Coordinator.

8. The Transaction Coordinator writes a *prepare commit* record to the transaction topic. At this point the transaction will complete no matter what failure scenarios may happen.

9. The Transaction Coordinator writes a *commit marker* to the outbound topic, changelog topic, and consumer offsets topic. Downstream transactional consumers (configured as *READ_COMMITTED*) will block until the *commit marker* is written to the topic partition.

10. The Transaction Coordinator writes a *complete commit* transaction record to the transaction topic. The producer can now begin its next transaction.

When the Kafka Streams application first starts the embedded producer registers a transaction Id with the Transaction Coordinator. The Transaction Coordinator assigns it a unique Producer Id, and writes this in an init record to the transaction topic.

Prior to the writes to the outbound and changelog topics, and prior to the write to the consumer offsets topic, the Kafka Streams library also sends an add partition request to the Transaction Coordinator. In both cases the Transaction Coordinator writes a record to the transaction log for these partitions inclusion in the transaction.

The initialise transactions request and the add partition requests are left off the above diagram for clarity, however are included in the sequence diagram below showing the full flow.

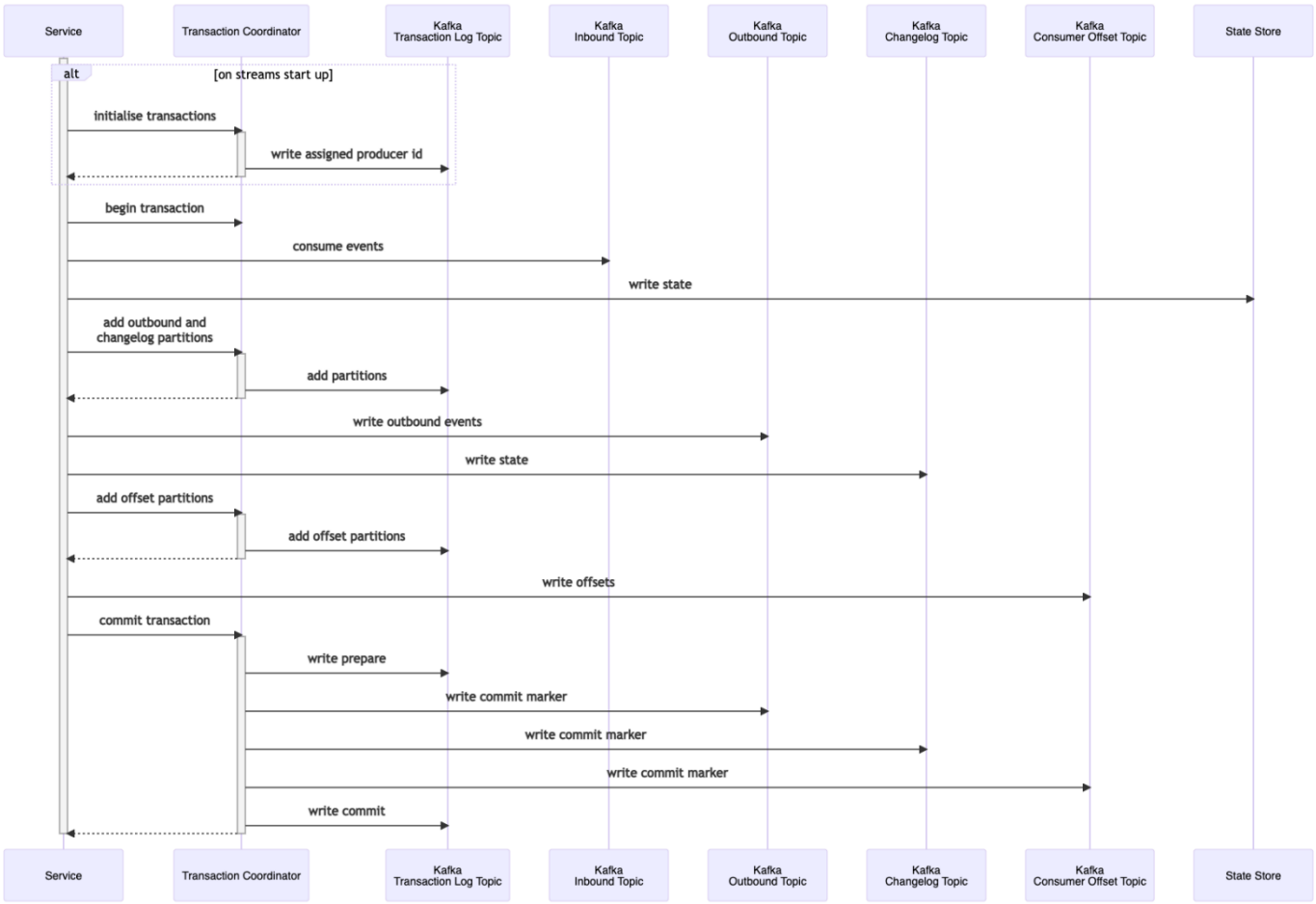

Figure 2: Transactional Kafka Streams flow - sequence diagram

If any errors happen during the stream processing, then an *abort transaction* request is sent to the Transaction Coordinator. The Transaction Coordinator then writes a *prepare abort* record to the transaction topic. It writes an *abort marker* to each of the outbound, changelog, and consumer offsets topics. Downstream transactional consumers can now skip these records. Finally the Transaction Coordinator writes an *abort complete* transaction record to the transaction topic.

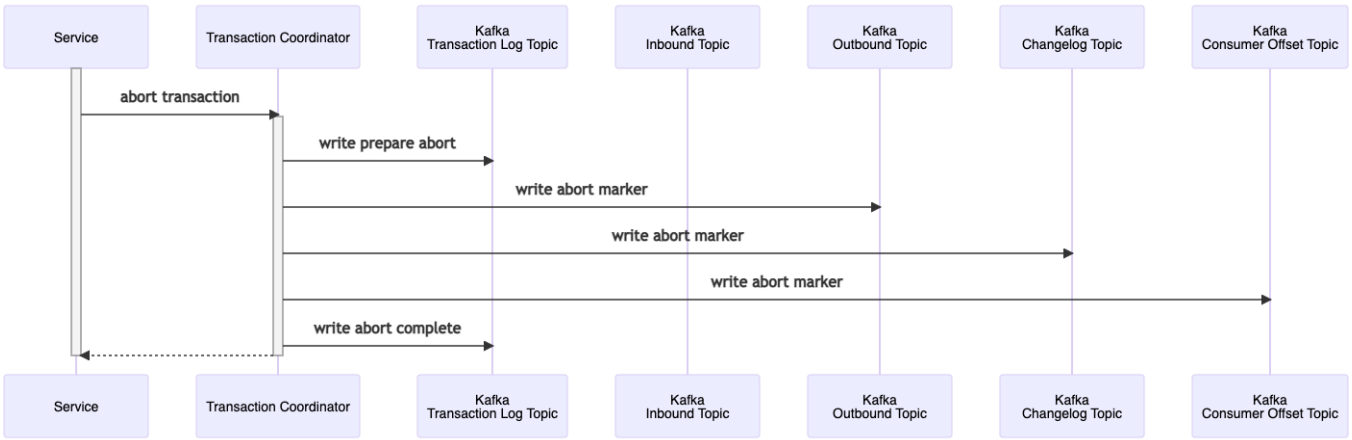

Figure 3: Abort commit

State Store

Changelog Topics

Kafka Streams state stores are detailed in this article Kafka Streams: State Store. In particular it covers the usage of changelog topics, which are an important factor when considering exactly-once messaging and ensuring the consistency and durability of the persisted state. Changelog topics back the state store, providing resilience for the store in failure scenarios. They are similar to write-ahead logs for databases, providing a durable source of truth that can be replayed to rebuild state.

The state that is written to the state store is also written to the changelog topic as part of the Kafka Streams processing flow. Kafka Transactions ensure that the writes to the changelog topic are atomic with the marking of the original message as successfully consumed (on the consumer offsets topic) and with any writes to outbound topics. i.e. these topic writes either all succeed or all fail. The messages on this topic are then replayed when an application is restarting and the state store is considered corrupt in order to rebuild the full application state.

Checkpoint File

With the default state store RocksDB, a checkpoint file is used to facilitate the ability of an application to recover and rebuild state following a restart. The Kafka Streams: State Store article covers this file in detail. Essentially it tracks the offsets of messages that have been written to the state store. If an application fails and needs to rebuild its state in a non-transactional flow the messages can be replayed from the changelog topic from the offsets retrieved from the checkpoint file.

With Kafka Transactions enabled in a Kafka Streams application, the way this file is used by the application differs. The checkpoint file is only written to on a clean task shutdown, such as happens when a consumer group rebalances. Once the task has been assigned to a consumer then the file is deleted before the consumer starts processing messages. Therefore the presence of the file means that the RocksDB state store is valid and up to date. If on task assignment following a restart there is no checkpoint file present it means that the application had crashed and the state store is very likely corrupt. The state store is deleted and rebuilt from the changelog topic.

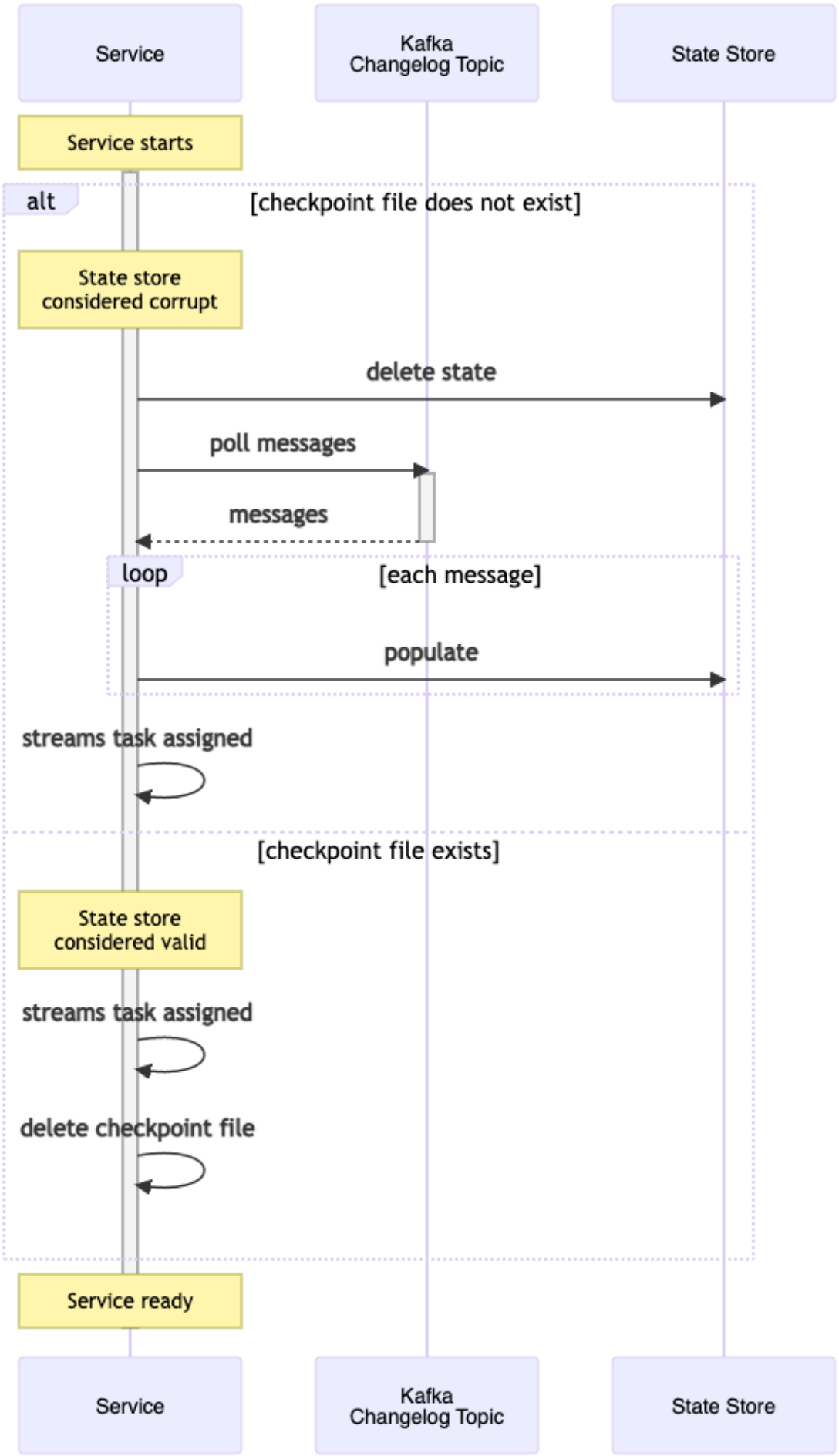
Service
Kafka Changelog Topic
State Store
Service starts
alt
[checkpoint file does not exist]
State store considered corrupt
delete state
poll messages
messages
loop
[each message]
populate
streams task assigned
[checkpoint file exists]
State store considered valid
streams task assigned
delete checkpoint file
Service ready
Service
Kafka Changelog Topic
State Store

Figure 4: Rebuilding the state store

Failure Scenarios

Kafka Streams are resilient to failures. Non-transactional streams ensure at-least-once delivery, so data is guaranteed not to be lost. However in failure scenarios duplicates can occur, both duplicate writes to the state store and changelog topics, and duplicate writes to the outbound topic. This could happen for example where a failure occurs after the write to the state store and the writes to the outbound and changelog topics have occurred but before the consumer offsets are written. The result is that the message will be redelivered on the next consume, and so a duplicate write to the state store and a duplicate write to the outbound topic would occur.

With transactions enabled, exactly-once messaging is guaranteed. As well as guaranteeing no data loss, duplicate writes to the state store and outbound topics are protected against. The following diagram illustrates a Kafka Streams task failing after writing to the state store and between writes to the topics. Each task has a transaction Id that is provided as a configuration parameter by the producer application. When the task is restarted it calls the Transaction Coordinator with an initialise transactions request, passing this transaction Id. The Transaction Coordinator aborts any associated dangling transaction, with abort markers being written to the topics. The task can now retrieve the consistent state from the changelog topic and rebuild the state store. It will then consume the next batch of events from the last committed offset on the consumer offsets topic, ensuring the messages are processed exactly once.

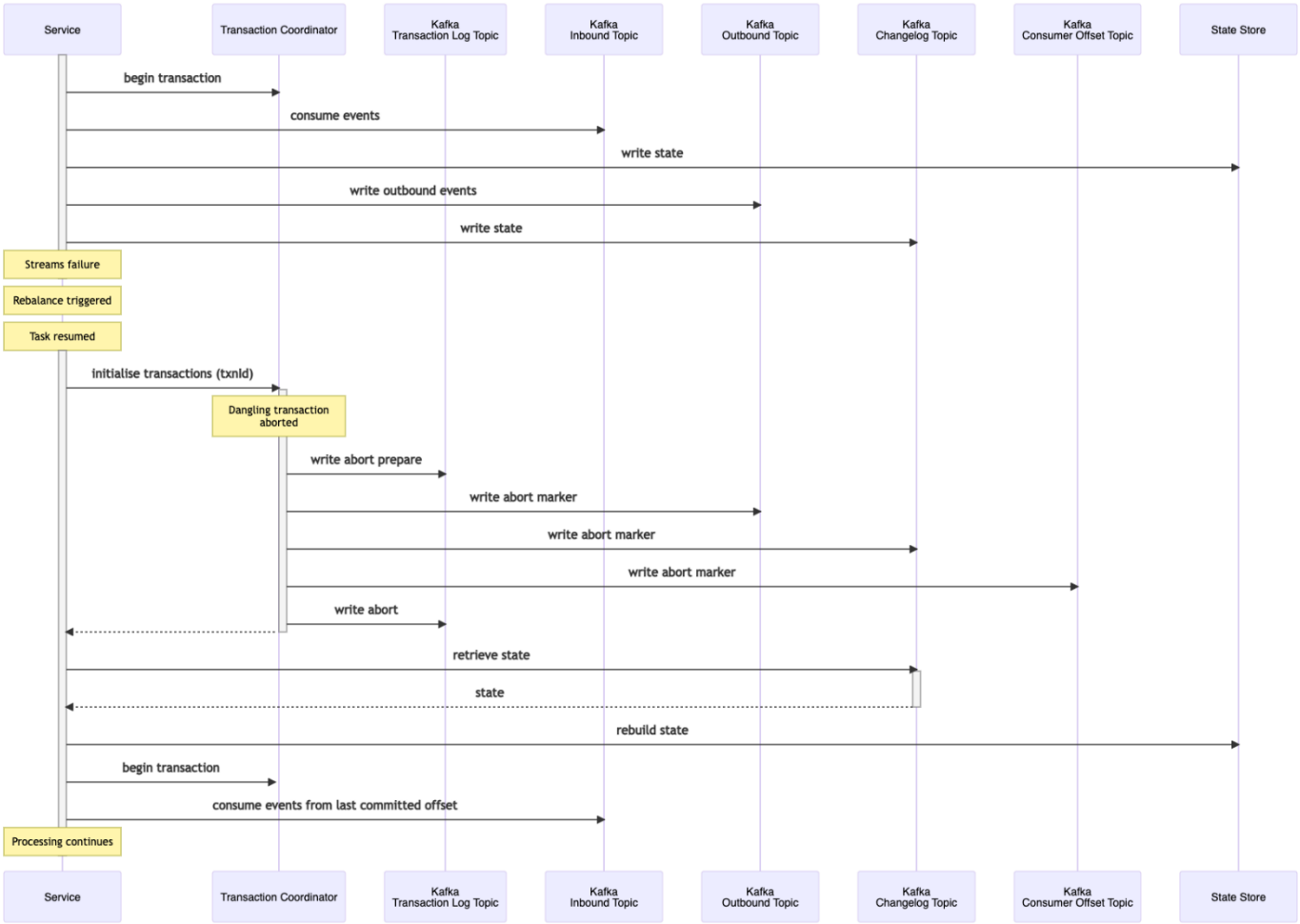

Figure 5: Transaction recovery on Streams failure

As stated previously, once the Transaction Coordinator has written the *prepare commit* entry to the transaction log, the transaction will complete (either as completed or aborted) whatever failure scenario occurs. If the Transaction Coordinator were itself to fail at the point after writing the *prepare commit* transaction entry on the transaction log,

when it restarts it first reads the transaction log to determine if there are any unfinished transactions. Any transactions it finds that are prepared for commit will be either completed or aborted as necessary.

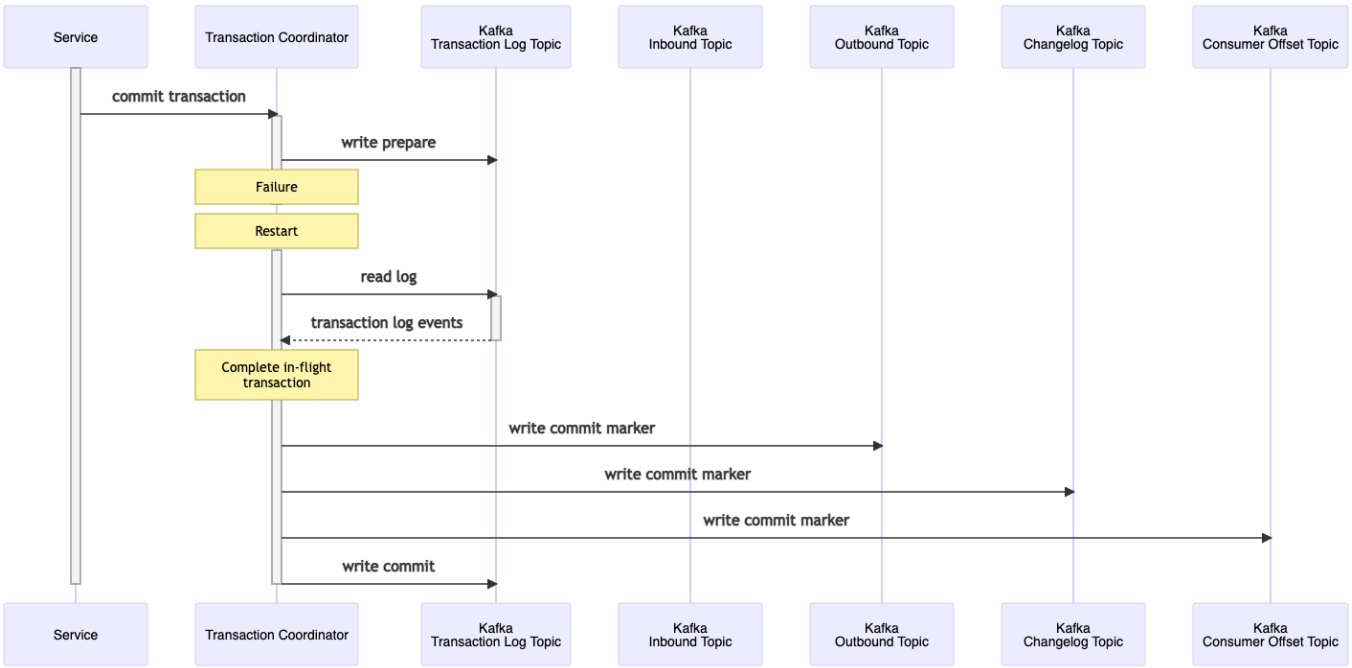

Figure 6: Transaction recovery on Transaction Coordinator failure

Performance

The trade-off for configuring exactly-once messaging is the impact to performance. There is a time cost to each transaction, given the extra processing and log writes that take place. Fewer, larger, transactions therefore have a smaller impact on throughput than many, smaller, transactions.

The overall latency increases as downstream transactional consumers can only consume messages that have been committed, and until that happens those consumers are blocked. Latency can be improved by shortening the **commit.interval.ms** configuration parameter resulting in more frequent, smaller transactions, at the cost of throughput. A tuning exercise is therefore required to find the optimal configuration and trade-off between throughput and latency.

As a rough approximation, something in the region of a 3% degradation in performance could be considered likely when enabling exactly-once messaging. However there are many factors in play, not least the transaction size, that would influence the level of degradation.

Configuration

In order to enable transactions the **processing.guarantee** configuration parameter should be set to **exactly_once** in Kafka Streams 2.x. In Kafka Streams 3.x a new version that improves the performance and scalability of partitions/tasks was introduced: **exactly_once_v2**. By default it is set to **at_least_once**. Exactly once processing requires a cluster with at least three broker nodes.

Conclusion

Enabling Kafka Transactions on a Kafka Streams application simply requires the addition of a single configuration parameter. With this, exactly-once messaging is guaranteed. Data remains consistent between the state store and Kafka topics no matter what failure scenarios occur. The risk of data loss and the need to cater for duplicate messages is removed. The trade-off for enabling exactly-once messaging that must be considered is the impact to

performance. Transaction sizes can be tuned to get the optimal balance between impact to throughput and latency.

More On Kafka Streams...

- The Kafka Streams: Introduction article delves into the API and covers its benefits and characteristics.
- The Kafka Streams: Spring Boot Demo article details the application that is under test.
- The Kafka Streams: Testing article examines the options and tools available to comprehensively test a Kafka Streams application.
- The Kafka Streams: State Store article looking at the role and usage of the state store in a stateful Kafka Streams application.

Resources

- Consistency and Completeness: Rethinking Distributed Stream Processing in Apache Kafka - Presentation by Guozhang Wang